



Formation Kubernetes

Session 9 — Terraform Avancé : Applications et Helm

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Recap Session 8

Modules Terraform, cluster provisioning, multi-cloud

10 min



Provider kubernetes

Gérer les ressources K8s depuis Terraform

15 min



Provider helm

Déployer des Helm charts depuis Terraform

10 min



Module app/

Le pattern générique : Deployment + Service + Ingress

15 min



Pièges Terraform + K8s

Provider dependency, state coupling, secrets, drift

10 min



Démo live & TP

Ajouter une app en ~20 lignes, créer votre module

1h

Recap — Session 8 : Modules

Terraform



Modules

Code réutilisable : network/, cluster/, node_pool/



GKE Cluster

Contrôle plane managé, node pool autoscaling (e2-small spot)



Multi-cloud ready

Même code = déployer sur GCP, AWS, Azure



State file

terraform.tfstate = source de vérité, à sécuriser & versionner

Question : comment déployer l'APPLICATION maintenant qu'on a l'infra ?

Provider kubernetes – Gérer K8s depuis Terraform

```
host          = google_container_cluster.primary.endpoint
token         = data.google_client_config.default.access_token
cluster_ca_certificate = base64decode(
  google_container_cluster.primary.master_auth[0].cluster_ca_certificate
)
```

Créer un namespace

```
metadata {
  name = "production"
}
```

Authentification : credentials du cluster → fichier kubeconfig traduit en Terraform

Provider helm – Déployer des Helm charts

```
kubernetes {  
  host          = google_container_cluster.primary.endpoint  
  token         = data.google_client_config.default.access_token  
  cluster_ca_certificate = base64decode(...)  
}
```

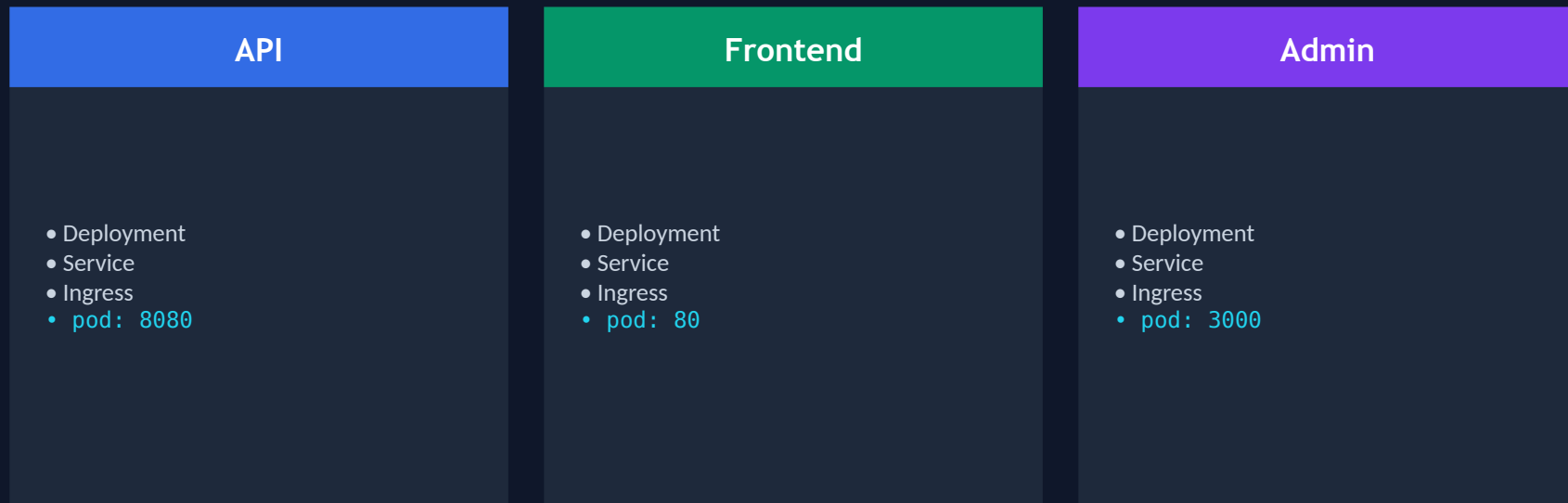
Déployer Traefik

```
name          = "traefik"  
repository    = "https://traefik.github.io/charts"  
chart         = "traefik"  
version       = "27.0.0"  
namespace     = kubernetes_namespace.traefik.metadata[0].name  
values        = [file("${path.module}/traefik-values.yaml")]
```

Helm charts depuis Terraform : valeurs en HCL ou fichiers YAML

Module app/ — Le pattern générique

Un seul module, N applications



module.app

Paramètres : app_name, image, replicas, port, env_vars, host, enable_ingress

Module traefik/ – Ingress Controller en Terraform

module.traefik – Tâches

1. Namespace traefik
2. helm_release Traefik
3. Service LoadBalancer
4. GCP NEG + firewall
5. Helm values : TLS, dashboard

NEG (Network Endpoint Group)

GCP integration

Traefik pods = NEG endpoints

Cloud LB voit les pods directement

Health checks via kubelet

Pas de Service node ports

Avantage : latence basse + scaling aisé

Secret Manager CSI – Intégrer GCP Secrets

Flux des secrets



```
metadata {  
  name = "db-credentials"  
}  
spec {  
  provider = "gcp"  
  parameters {  
    secrets = jsonencode({  
      "1" : { resourceName : "projects/.../.../db-password" }  
    })  
  }  
}
```

Secrets = montés en volumes dans les pods, jamais dans etcd K8s

Developer Experience – Makefile patterns

make init

Auth GCP + backend + terraform init

make plan

terraform plan → génère tfplan

make apply

terraform apply tfplan

**make traefik-
dashboard**

Port-forward vers dashboard Traefik (localhost:8080)

make state-rm

Supprimer une ressource du state

make state-import

Importer une ressource existante

DX matter : une bonne CLI = adoption par les équipes

Architecture du TP

3 stagiaires, 1 cluster GKE partagé, un namespace par stagiaire

Cluster GKE partagé — europe-west9 | provisionné par le formateur

Traefik (singleton) | Namespace: traefik

Namespace: alice

- API Deployment
- Frontend Deployment
- 2x Services
- 2x Ingress

Namespace: bob

- API Deployment
- Frontend Deployment
- 2x Services
- 2x Ingress

Namespace: charlie

- API Deployment
- Frontend Deployment
- 2x Services
- 2x Ingress

Étapes du TP : 1) Ajouter apps au main.tf | 2) terraform plan/apply | 3) Tester les routes

Pièges Terraform + Kubernetes

Provider dependency

kubernetes + helm providers ont besoin du cluster. Dépendance circulaire si on déclare mal.

Prévention :

depends_on, ou séparer en 2 stacks (infra / apps)

State coupling

terraform.tfstate contient l'état du cluster ET des apps. Supprimer une ressource = risky.

Prévention :

state rm + reimport souvent. Monolith = risque.

Secrets en plaintext

Si on met des secrets dans les variables Terraform, ils se retrouvent en plaintext dans tfstate.

Prévention :

Toujours utiliser GCP Secret Manager + CSI driver. Ne JAMAIS terraform var pour secrets.

Drift

Quelqu'un modifie une ressource avec kubectl → tfstate désynchronisé. Plan dit 'no changes' mais c'est faux.

Prévention :

Mettre des alertes, lancer terraform plan automatiquement, ou forbid kubectl changes (RBAC)

Démo live — Infrastructure RFLKT

Infra réelle : staging/main.tf avec 6 instances du module app



module.app 'api'

API Go, 2 replicas, internal + external routing



module.app 'frontend'

React, nginx, frontend.reflekt.io



module.secret_manager_csi

Secrets mappés depuis GCP Secret Manager



module.traefik

Ingress controller déployé via Helm



module.cloudflare_dns

DNS records créés automatiquement



module.gcp_managed_certs

SSL certificates gérés par GCP

terraform apply : chaque module ajoute 20-50 lignes. Zéro "config driftée"



Récapitulatif



Provider kubernetes + helm = déployer apps depuis Terraform



Module app/ = pattern réutilisable pour N'IMPORTE QUELLE app



CSI driver = secrets sécurisés depuis GCP Secret Manager



Traefik + NEG = ingress controller performant sur GCP



DX = Makefile abstrait la complexité Terraform

Prochaine étape — Session 10 : HPA, Probes & Production Patterns

Autoscaling horizontal, sondes liveness/readiness/startup, requests/limits — et comment tout câbler en Terraform.